

CyberLab Quick Reference

Sep 27, 2026 · @Saaaid

Where you run everything

Every command runs on your **Ubuntu Server host**, in the same terminal (or SSH session) where `sudo docker ps` works. There is no second machine.

Two layers are easy to mix up:

- **The host** is your Ubuntu Server. You run `cyberlab`, `docker`, and `tcpdump` here.
- **The attacker box** is a Kali *container* running on the host. Its shell is where you run `nmap`, `sqlmap`, `hydra`, and `nuclei`. You do not need a Kali machine — Kali is just the attack container's image.

The one thing to remember: `cyberlab` manages the lab (start, check, scan, reset); a shell inside the attacker container is where you run the actual attack tools. Get into that shell with `sudo docker exec -it attacker bash`.

cyberlab command cheat sheet

Run these on the host with `sudo`.

Command	What it does
<code>cyberlab up</code>	Start the whole lab (first run downloads images)
<code>cyberlab status</code>	List the containers and whether each is up
<code>cyberlab verify</code>	Self-test: segmentation, services, IDS, Nuclei installed
<code>cyberlab scan <target></code>	Run a Nuclei vulnerability scan, e.g. <code>cyberlab scan http://10.20.0.12</code>
<code>cyberlab alerts</code>	Live Suricata IDS alerts (Ctrl+C to stop)
<code>cyberlab rules</code>	Download/refresh the Suricata rule set
<code>cyberlab reset</code>	Rebuild the target containers, keep rules and logs
<code>cyberlab reset --full</code>	Rebuild everything, including rules and Portainer data
<code>cyberlab down</code>	Stop the lab (keeps everything)

Command	What it does
<code>cyberlab destroy</code>	Remove containers, volumes and images

Start of a session: `sudo cyberlab up` then `sudo cyberlab verify`. End of a session: `sudo cyberlab down`.

The lab map

Three segments: Internet, DMZ and LAN. The attacker sits on the Internet; targets sit in the DMZ and LAN.

Host	IP	Segment	Notes
attacker (Kali)	10.10.0.10	Internet	Your attack box
router	10.10.0.254 / 10.20.0.254	Internet + DMZ	Firewall between Internet and DMZ
juiceshop	10.20.0.11:3000	DMZ	Web target
dvwa	10.20.0.12:80 (also 10.30.0.12)	DMZ + LAN	Web target and the pivot into the LAN
metasploitable	10.30.0.20	LAN	Internal target, no direct route from the attacker

What can reach what:

- The attacker reaches the **DMZ** on TCP 80 and 3000 and ICMP only — everything else is blocked by the router.
- The attacker has **no direct route to the LAN** (10.30.0.20). You reach Metasploitable only by pivoting through DVWA, which is dual-homed on both the DMZ and the LAN.
- The DMZ and LAN have **no internet access** — by design.

The attack box and its tools

Get a shell inside the attacker container:

```
sudo docker exec -it attacker bash
```

Everything after this runs *inside* that shell. Type `exit` to leave.

Pre-installed in the attacker container: `nmap`, `sqlmap`, `hydra`, `nuclei`, `curl`, `netcat (nc)`, `ping`, and `ip`. Install anything else with `apt-get update && apt-get install -y <tool>` — note that additions live only until the container is rebuilt.

To browse the web targets from a browser on the host, point it at the SOCKS5 proxy `127.0.0.1:1080`, then open `http://10.20.0.11:3000` (Juice Shop) or `http://10.20.0.12` (DVWA). Portainer's Docker GUI is at `https://localhost:9443`.

Task workflows

Run the tool commands from inside the attacker shell (`sudo docker exec -it attacker bash`) unless noted.

1. Recon with nmap. Scan the DMZ from the attacker box:

```
nmap -sV 10.20.0.11-12      # service/version scan of the web targets
nmap -sV --script vuln 10.20.0.12  # add vuln scripts against DVWA
```

The attacker cannot scan the LAN target (10.30.0.20) directly — that is expected, and is what the pivot step is for.

2. Vulnerability scan with Nuclei. From the host:

```
sudo cyberlab scan http://10.20.0.12      # DVWA
sudo cyberlab scan http://10.20.0.11:3000  # Juice Shop
```

Record the finding count by severity as your before/after number.

3. Web exploitation. Browse DVWA (`http://10.20.0.12`) or Juice Shop (`http://10.20.0.11:3000`) through the SOCKS proxy, or drive them from the shell:

```
sqlmap -u "http://10.20.0.12/vulnerabilities/sqli/?id=1" --batch --cookie="<your session cookie>"
hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.20.0.12 http-post-form "..."
```

4. Pivot to the LAN (Metasploitable). Reach 10.30.0.20 through DVWA, which is dual-homed. Confirm the pivot host sees it, then work from there:

```
sudo docker exec dvwa bash -c 'timeout 3 bash -c "</dev/tcp/10.30.0.20/21" && echo FTP-open'
```

For a full workflow, use DVWA's command injection to run recon commands against 10.30.0.20, or set up a proxy/port-forward through DVWA and point nmap at the LAN through it.

Packet capture and Wireshark

The attacker container has no GUI, so capture on the **host** with `tcpdump`, then open the saved file in Wireshark. Each lab segment is a host bridge: `br-inet` (Internet), `br-dmz` (DMZ), `br-lan` (LAN).

Capture traffic to a file (run on the host):

```
sudo tcpdump -i br-inet -w /tmp/inet.pcap      # attacker <-> DMZ traffic
sudo tcpdump -i br-lan -w /tmp/lan.pcap        # pivot <-> Metasploitable traffic
```

Stop with `Ctrl+C`, then open the `.pcap` in Wireshark:

- If Wireshark is on the same Ubuntu host with a desktop: `wireshark /tmp/inet.pcap`.
- If Wireshark is on your laptop: copy the file over first, e.g. `scp kali@<server-ip>:/tmp/inet.pcap .`, then open it in Wireshark.

To watch live in the shell instead of Wireshark, add a filter: `sudo tcpdump -i br-dmz -n port 80 or port 3000`. For a quick capture from the attacker's own view, install `tshark` in the container (`apt-get install -y tshark`) and run `tshark -i eth0`.

Watching IDS alerts (Suricata)

Suricata sniffs the Internet and LAN bridges and alerts on the protected segments (DMZ and LAN). Watch alerts live while you run an attack in another terminal:

```
sudo cyberlab alerts # tails fast.log; Ctrl+C to stop
```

For the full JSON event stream:

```
sudo docker exec suricata tail -f /var/log/suricata/eve.json
```

A good exercise: run your nmap or sqlmap step, then find the matching alert in `cyberlab alerts` and note the signature. If you see no alerts, load the rules first with `sudo cyberlab rules` and try again. Endpoint-only techniques often go undetected here because the lab has network sensors only — that gap is worth discussing.

Reset, cleanup and safety

- **Broke a target?** `sudo cyberlab reset` rebuilds the target containers from clean images and keeps your rules and logs.
- **Fresh start of everything:** `sudo cyberlab reset --full` (also clears Suricata rules and Portainer data).
- **Done for now:** `sudo cyberlab down` stops the lab; `sudo cyberlab up` brings it back.

Safety rules:

- Attack only the lab targets: 10.10.0.0/24, 10.20.0.0/24 and 10.30.0.0/24. Never point these tools at anything else.
- Nothing is published beyond the host: only 127.0.0.1:9443 (Portainer) and 127.0.0.1:1080 (SOCKS). Keep it that way.
- Run exploit tools inside the `attacker` container, never on the Ubuntu host itself.